Introduction to Express Framework

Definition:

Express.js is a fast, minimal, and flexible web application framework for Node.js. It provides useful features to develop web applications and APIs easily. It simplifies server creation, request handling, routing, and response management.

Features of Express Framework:

1. **Simple and Lightweight: Express provides a simple structure for developing web applications with less code.**
2. **Routing: Express allows developers to define routes to handle different URLs and HTTP requests, such as GET and POST.**
3. **Request and Response Handling: It provides methods to receive client requests, process data, and send responses to clients.**
4. **Middleware Support: Middleware functions can execute code, process requests, modify request or response objects, and perform other tasks during the request-response cycle.**
5. **Static File Handling: Express can serve static files such as HTML pages, CSS files, JavaScript files, and images.**
6. **API Development: Express makes it easier to develop REST APIs that allow frontend applications to communicate with servers.**
7. **Template Engine Support: Express supports template engines that generate dynamic HTML pages using data from the server.**

Advantages of Express Framework:

1. **It reduces the amount of code required for server-side development.**
2. **It makes routing and request handling easier.**
3. **It supports middleware for organizing application functionality.**
4. **It works with Node.js and its large ecosystem of packages.**
5. **It is suitable for developing web applications and REST APIs.**

Real-Time Example:

In an online shopping application, Express can handle requests to display products, accept customer login details, and process orders. For example, when a user requests a product page, Express matches the requested URL to a route and sends the appropriate response.

Express Server Request Response Routes

Definition:

Express.js is a web application framework for Node.js that simplifies handling client requests and sending server responses. Routes define how the server responds to requests made to a particular URL using a specific HTTP method.

1. Express Server Request-Response Cycle:

The request-response cycle describes how a client sends a request to the server and receives a response.

1. **Client Request: A client, such as a web browser, sends an HTTP request to the Express server.**
2. **Request Reception: The Express server receives the request and checks its URL and HTTP method.**
3. **Request Processing: Express matches the request with the appropriate route and executes its handler function.**
4. **Response Generation: The handler processes the request and prepares the required response.**
5. **Response Sent: The server sends the response, such as HTML, JSON data, or a text message, back to the client.**

2. Routes in Express:

Routes determine how an application responds to a particular URL and HTTP request method. Important HTTP methods used in routing are:

1. **GET: Used to retrieve data from the server.**
2. **POST: Used to submit data to the server.**
3. **PUT: Commonly used to update or replace an existing resource.**
4. **DELETE: Used to delete a resource.**

3. Working of Routes:

1. **The client sends a request to a particular URL.**
2. **Express checks the HTTP method and URL.**
3. **If a matching route is found, its handler function executes.**
4. **The handler processes the request and sends an appropriate response.**
5. **If no matching route or other response mechanism handles the request, Express normally returns a 404 Not Found response.**

Real-Time Example:

In a student management system, when a student requests their profile, the browser sends a GET request to the server. Express matches the profile route, retrieves the student's details, and sends them back to the browser. When the student submits registration details, a POST route can process the submitted information.

Advantages:

1. **It makes request handling simple and organized.**
2. **It allows different URLs to perform different operations.**
3. **It supports multiple HTTP methods.**
4. **It helps developers build structured web applications and REST APIs.**

Route Parameters

Definition:

Route parameters are named values included in the URL that are used to capture specific information from a client's request. In Express.js, route parameters are defined using a colon (:) followed by a parameter name. They are available in the req.params object.

Working of Route Parameters:

1. **Define a Route: The developer creates a route with a named parameter in the URL.**
2. **Receive a Request: The client sends a request to that URL with a specific value.**
3. **Match the Route: Express matches the requested URL with the defined route.**
4. **Extract the Parameter: Express extracts the value from the URL and stores it in the req.params object.**
5. **Process the Request: The route handler uses the extracted value to perform the required operation.**
6. **Send a Response: The server sends the requested information back to the client.**

Advantages:

1. **They allow dynamic URLs instead of creating separate routes for every item.**
2. **They make it easy to identify a specific resource.**
3. **They simplify request handling in web applications.**
4. **They are useful for developing REST APIs.**

Real-Time Example:

Suppose a student management website stores details of many students. When a teacher wants to view student number 105, the browser requests /students/105. Express uses the route parameter 105 to identify the requested student and return their details.

Short Example:

```
app.get('/students/:id', (req, res) => {
  res.send('Student ID: ' + req.params.id);  });
```

Here, :id is the route parameter. If the URL is /students/105, then req.params.id contains 105.

Multiple Route Callback/Handler Functions

1.Definition

Multiple Route Callback/Handler Functions in Node.js Express are used to execute multiple callback functions for a single route. These functions perform different tasks when a client sends a request to the server.

2. Explanation

- **A route defines how the server responds to a client request.**
- **Multiple callback functions can be used for a single route.**
- **Each callback performs a specific task.**
- **The next() function passes control to the next callback.**
- **These functions help divide tasks into smaller and manageable parts.**
- **They are useful for authentication, validation, and processing requests.**

3. Syntax

```
app.get('/user',
    function(req, res, next) {
        console.log("First callback");
        next();
    },
    function(req, res) {
        res.send("Second callback executed");
    }
);
```

4. Example

```
const express = require('express');
const app = express();

app.get('/home',
    function(req, res, next) {
        console.log("Welcome to Home");
        next();
    },
    function(req, res) {
        res.send("Hello User!");
    }
);

app.listen(3000);
```

Output:

- **On the server console: Welcome to Home**
- **On the browser: Hello User!**

5. Advantages

1. **It divides tasks into smaller functions.**
2. **It improves code reusability.**
3. **It makes code easier to maintain.**
4. **It supports request validation and authentication.**

Methods of Response Object

1. Definition

In Node.js Express, the Response Object (res) is used by the server to send a response to the client after receiving a request.
It provides different methods to send data, display messages, send files, and set response status codes.

2. Important Methods of Response Object

1. **res.send() – Sends a response to the client.**
2. **res.json() – Sends data in JSON format.**
3. **res.status() – Sets the HTTP status code.**
4. **res.redirect() – Redirects the client to another URL.**
5. **res.sendFile() – Sends a file to the client.**
6. **res.render() – Renders a view or template.**
7. **res.end() – Ends the response process.**

3. Example

```
const express = require('express');
const app = express();
app.get('/', (req, res) => {
    res.send("Welcome to Express");
});
app.get('/user', (req, res) => {
    res.json({ name: "Rahul", age: 22 });
});
app.get('/error', (req, res) => {
    res.status(404).send("Page Not Found");
});
app.listen(3000);
```

Output:

- **/ → Welcome to Express**
- **/user → {"name":"Rahul","age":22}**
- **/error → Page Not Found (HTTP status 404)**

4. Advantages

- **Sends different types of responses to clients.**
- **Makes response handling easy.**
- **Supports JSON data and file transmission.**
- **Helps manage status codes and redirection.**

5. Conclusion

The Response Object is an important part of Express.js. Its methods help the server send data, files, status codes, and messages to the client.

Chaining Route Handlers

1. Definition

In Node.js Express, Chaining Route Handlers means using multiple callback functions for the same route to handle a client request.
These functions execute one after another using the next() function.

2. Explanation

- **Route handlers define how the server responds to a request.**
- **Multiple handlers can be attached to a single route.**
- **Each handler performs a specific task.**
- **The next() function passes control to the next handler.**
- **The final handler sends the response to the client.**
- **It helps organize code and makes it easier to maintain.**

3. Syntax

```
app.get('/user',
    function(req, res, next) {
        // First handler
        next();
    },
    function(req, res) {
        // Second handler
        res.send("User Page"); }
);
```

4. Example

```
const express = require('express');
const app = express();

app.get('/student',
    (req, res, next) => {
        console.log("First Handler");
        next();  },
    (req, res) => {
        res.send("Welcome Student");
    }  );
app.listen(3000);
```

Output:

- **Server console: First Handler**
- **Browser: Welcome Student**

Explanation: First, the first handler prints a message in the console. Then, next() transfers control to the second handler, which sends the response to the browser.

5. Advantages

1. **It divides tasks into smaller functions.**
2. **It improves code readability.**
3. **It makes code easier to maintain.**
4. **It supports validation and authentication.**
5. **It allows multiple operations on the same route.**

Send Static Files

1. Definition : In Node.js Express, sending static files means sending files such as HTML, CSS, JavaScript, images, and other resources directly to the client.Express provides the express.static() middleware to serve static files.

2. Explanation

- **Static files do not change dynamically for each request.**
- **Examples include HTML pages, CSS files, images, and JavaScript files.**
- **Express uses express.static() to make these files available to clients.**
- **Static files are usually stored in a folder named public.**
- **The browser can access these files through a URL.**

3. Syntax : `app.use(express.static('public'));`

Here, public is the folder containing static files.

Folder structure:

```
project/
 ├── app.js
 └── public/
     ├── index.html
     ├── style.css
     └── image.jpg
```

4. Example

```
const express = require('express');
const app = express();
app.use(express.static('public'));
app.listen(3000);
```

Output : If the public folder contains index.html, open:
http://localhost:3000/index.html
The browser displays the HTML page.
Similarly, the CSS file can be accessed at:
http://localhost:3000/style.css

5. Advantages = 1.It serves files easily. 2.It reduces the need to write separate routes for each file. 3. It supports HTML, CSS, JavaScript, and images. 4. It makes web application development easier.

Manage Cookies

1. Definition : Cookies are small pieces of data stored in the user's web browser. They help a website remember information about a user between requests.In Express.js, cookies can be managed using the cookie-parser middleware.

2. Explanation

- **Cookies store small amounts of information in the browser.**
- **They help maintain user sessions and remember user preferences.**
- **The server can create cookies and send them to the browser.**
- **The browser sends cookies back to the server with later requests.**
- **Cookies can be read and deleted using Express.js methods and middleware.**

3. Important Methods

1. **res.cookie() – Creates a cookie and sends it to the browser.**
2. **req.cookies – Reads cookies received from the browser using cookie-parser.**
3. **res.clearCookie() – Clears a cookie from the browser.**

4. Example : First, install the required package:
npm install express cookie-parser

5. Advantages

1. **Stores user preferences.**
2. **Helps remember users between requests.**
3. **Supports session management. 4. Improves the user experience.**

Accept User Input

1. Definition

Accepting user input means collecting data entered by a user through a form, keyboard, or other input method and processing it on the server.
In Node.js Express, user input can be received through req.query, req.params, and req.body.

2. Methods to Accept User Input

1. **req.query – Used to get data from the URL query string.**
2. **req.params – Used to get values from route parameters.**
3. **req.body – Used to get data submitted through forms or request bodies.**

3. Example

The following example accepts a user's name using a query string.

```
const express = require('express');
const app = express();
app.get('/user', (req, res) => {
    const name = req.query.name;
    res.send("Hello " + name);
});
app.listen(3000);
```

Output:

Open this URL in the browser:

http://localhost:3000/user?name=Rahul

Output:

Hello Rahul

Explanation: The server reads the name using req.query.name and sends a greeting to the browser.

4. Example Using req.body

```
const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true }));
app.post('/user', (req, res) => {
    res.send("Hello " + req.body.name);
});
app.listen(3000);
```

Here, req.body.name reads the name submitted in a form using the POST method. A form with an input field named name is required.

5. Advantages

1. **It allows users to submit information.**
2. **It helps process form data.**
3. **It supports different types of requests.**
4. **It makes web applications interactive.**

6. Conclusion

Accepting user input is an important feature of Node.js Express. It allows the server to receive and process user data using req.query, req.params, and req.body.

File Upload with Express

1. Definition

File Upload with Express.js means uploading files such as images, documents, and PDFs from a client to a server using Node.js and Express.

Express can handle file uploads with the help of middleware such as Multer.

2. Explanation

- **The user selects a file using an HTML form.**
- **The form sends the file to the server using the POST method.**
- **Multer processes the uploaded file.**
- **The server stores the file in a specified folder.**
- **The server sends a response to the user after uploading the file.**

3. Example

First, install Express and Multer.

```
npm install express multer
HTML Form (index.html)
<form action="/upload" method="POST"
      enctype="multipart/form-data">
    <input type="file" name="myfile">
    <button type="submit">Upload</button>
</form>
Node.js Code (app.js)
const express = require('express');
const multer = require('multer');
const app = express();
const upload = multer({ dest: 'uploads/' });
app.post('/upload', upload.single('myfile'),
    (req, res) => {
        res.send('File uploaded successfully');  }
);
app.listen(3000);
```

4. Output

- **The browser displays a file selection form.**
- **The user selects a file and clicks Upload.**
- **The server saves the file in the uploads folder.**
- **The browser displays: File uploaded successfully.**

5. Advantages

1. **It allows users to upload files easily.**
2. **It supports images, documents, and other files.**
3. **Multer simplifies file handling.**
4. **It is useful for profile pictures and document submission systems.**

6. Conclusion

File uploading is an important feature of web applications. Express.js uses middleware such as Multer to receive and store files on the server.

Send file as a response

1. Definition

In Node.js Express, sending a file as a response means sending a file from the server to the client when the client requests it.
Express provides the res.sendFile() method to send files such as HTML documents, images, and PDF files.

2. Explanation

- **The client sends a request to the server.**
- **The server locates the required file.**
- **The res.sendFile() method sends the file to the client.**
- **The browser displays the file or downloads it, depending on the file type and browser settings.**
- **An absolute file path is generally required.**

3. Syntax

res.sendFile(absoluteFilePath);

4. Example

```
const express = require('express');
const path = require('path');
const app = express();
app.get('/home', (req, res) => {
    res.sendFile(path.join(__dirname, 'index.html'));
});
app.listen(3000);
```

Explanation:

- **express is used to create the web server.**
- **path helps create the file path.**
- **__dirname represents the current directory.**
- **res.sendFile() sends index.html to the browser.**

5. Output

When the user opens:
http://localhost:3000/home
The browser displays the contents of index.html.

6. Advantages

1. **It sends files directly to clients.**
2. **It supports HTML pages, PDFs, and images.**
3. **It makes file delivery easy.**
4. **It reduces the need to manually read and send file contents.**

Templates and Express.

1. Definition

A template engine in Express.js is used to create dynamic HTML pages by combining HTML templates with data provided by the server.
Express supports template engines such as EJS, Pug, and Handlebars.

2. Explanation

- **Templates are used to generate dynamic web pages.**
- **They allow us to display data from the server in HTML pages.**
- **Express supports different template engines.**
- **The res.render() method is used to render a template.**
- **Templates reduce code repetition and make web development easier.**

3. Steps to Use Templates in Express

1. **Install a template engine.**
2. **Set the view engine in Express.**
3. **Create a views folder.**
4. **Create a template file.**
5. **Use res.render() to display the template.**

4. Example Using EJS

```
Step 1: Install EJS
npm install express ejs
Step 2: Create app.js
const express = require('express');
const app = express();
app.set('view engine', 'ejs');
app.get('/', (req, res) => {
    res.render('home', { name: 'Rahul' });
});
app.listen(3000);
Step 3: Create views/home.ejs
<h1>Welcome <%= name %></h1>
```

5. Output

When the user opens http://localhost:3000, the browser displays:
Welcome Rahul
Explanation: Express sends the name value to the EJS template. EJS inserts the value into the HTML page, and res.render() sends the generated page to the browser.

6. Advantages

1. **Creates dynamic web pages.**
2. **Reduces code repetition.**
3. **Makes code easier to maintain.**
4. **Supports displaying server-side data.**
5. **Separates HTML presentation from application logic.**